Unit I: Sorting & Graph Algorithms

Strongly Connected Components (SCC)

PPT PRESENTED BY

NAME : Srinivas Rao Tammireddy

ROLL NO : 257Y1D5805

YEAR : I Year II Semester

ACADEMIC YEAR : 2025 - 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

What is a Strongly Connected Component?

A Strongly Connected Component (SCC) of a directed graph G = (V, E) is a maximal set of vertices U ⊆ V such that for every pair of vertices u, v ∈ U, there is a directed path from u to v and a directed path from v to u.

Core Mathematical Properties

- **Connectivity equivalence:** The 'path connection' relation (u ~ v iff u is reachable from v and v from u) is an equivalence relation on V.
- **Partitioning:** The equivalence classes of this relation partition the graph G into disjoint SCCs.
- **Component Graph (DAG):** If we collapse each SCC into a single super-node, the resulting graph of components is a Directed Acyclic Graph (DAG).
- **Cycles:** An SCC contains at least one cycle (unless it is a single vertex with no self-loop).

Reachability & DFS Trees

- **DFS Tree structure:** Running Depth-First Search (DFS) on G partitions G into a forest of DFS trees.
- **Reachability Bounds:** If two vertices u and v belong to the same SCC, they are guaranteed to be in the same DFS tree during any search.
- **Transpose Graph G^T:** G^T is the same graph G but with all edge directions reversed. G and G^T have the EXACT same SCCs because reachability is symmetric.

Kosaraju's DFS-Based Algorithm

Kosaraju-Sharir algorithm computes all SCCs of a directed graph in O(V + E) time:

The Two-Pass DFS Algorithm Steps

1. Run First DFS on Graph G: Traverse G using DFS. As each vertex finishes execution (when all its outgoing edges are explored), push it onto an empty stack S.
2. Reverse Graph to Get G^T: Compute the transpose graph by reversing the directions of all edges in G.
3. Run Second DFS on G^T in Stack Order: While stack S is not empty:

- Pop the top vertex v from stack S.
- If v has not been visited in this second pass, it is the root of a new SCC.
- Run DFS starting at v in G^T. All vertices reached from v in G^T belong to the same SCC as v.
- Output this group as a completed SCC and mark them as visited.

4. Time Complexity: O(V + E) since it runs two standard DFS scans and one graph transposition.

Why It Works: Correctness Proof

Understanding the mathematical mechanism behind Kosaraju's algorithm:

The Leakage Problem & Transpose

• **Why not run DFS twice on G?** If we simply run DFS on G, the search will leak from one SCC to another connected downstream, merging them incorrectly.
• **The Transpose Shield:** In G^T, edges go backward. If there is a directed path SCC1 → SCC2 in G, then in G^T the path goes SCC2 → SCC1.
• **The Stack Order:** The first DFS finish order guarantees that the top node of the stack belongs to a 'source' SCC in G (meaning it has no incoming edges from other SCCs in G^T).
• **Clean Isolation:** When we run DFS in G^T starting at this popped root, it cannot escape to other components, isolating the SCC perfectly.

Kosaraju's vs. Tarjan's Algorithm

• **Tarjan's SCC Algorithm:** Another O(V + E) method that finds SCCs in a SINGLE DFS pass.
• **How Tarjan's Works:** Tracks DFS discovery numbers and the lowest reachable node index ('low-link' values) on an active stack.
• **Comparison:**

- **Kosaraju's:** Conceptually simpler, easier to implement, but requires transposing the graph and running two DFS passes.

- **Tarjan's:** More complex logic, but faster in practice since it only traverses the graph once and needs no transpose step.

Applications of SCCs

Strongly Connected Components model dependencies and structures in complex networks:

🌐 Social Network Communities

Identifies tightly knit groups of users where everyone interacts or shares info. Used to analyze information spread and recommend connections.

💻 Software Dependency Loops

Compilers build module import graphs. Finding SCCs reveals circular imports (e.g. A imports B, B imports C, C imports A) which must be resolved.

🔍 Web Page Ranking (Google PageRank)

The web forms a directed graph of links. PageRank partitions the web into SCCs to calculate page importances without getting stuck in link cycles.

⚙️ Deadlock Detection in DBMS

Database engines maintain 'wait-for' graphs of transactions waiting for locks. An SCC in this graph indicates a cycle, revealing a system deadlock.

THANK YOU

Department of Computer Science and Engineering
Marri Laxman Reddy Institute of Technology and Management